

CTF 2026

SIGNAL LOST

Digital Forensics & Incident Response — Challenge 495

CASE RESULT

Hidden dispatch recovered from a deceptive image artifact. The investigation followed the chain: challenge bundle → suspicious image → embedded signal → decoded archive → password → extracted dispatch → flag.

Challenge	Signal Lost
Category	DFIR
Points	495
Room by	THRYVE CTF (Aws Afaneh — Nova0x)
Final flag	Thryve{d0nt_trust_th3_f1l3_ext3ns10n_follow_th3_s1gnal}
Investigator	Het Dalal

1. Executive Summary

The challenge presented a bundle of recovered surveillance stills and stated that most were normal images while one concealed the final dispatch. The investigation therefore treated the image set as a file-analysis problem rather than relying only on visual inspection.

During analysis, one image was found to contain an embedded signal blob. The blob was extracted and decoded through Base64 and hexadecimal conversion to produce a 7z archive. A separate recovered PDF supplied the archive key after three successive Base64 decoding operations. The password-protected archive was then opened, revealing a README and a final dispatch text file. The README stated that the final line mattered; that line contained the accepted flag.

FINAL FINDING

Thryve{d0nt_trust_th3_f1l3_ext3ns10n_follow_th3_s1gnal}

2. Challenge & Evidence Scope

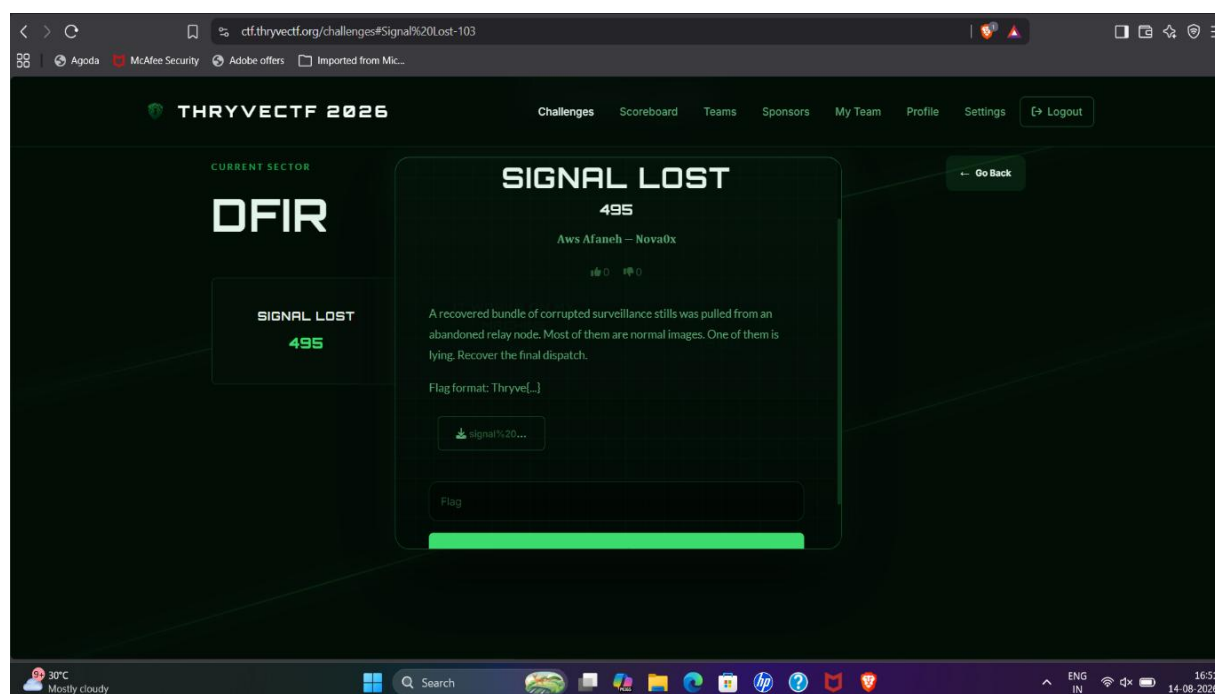


Figure 1 — Original challenge page. The prompt identifies the task as DFIR and states that one of the recovered surveillance stills is concealing the final dispatch.

The challenge wording is important because it establishes the investigative hypothesis: the useful evidence is likely inside one of the supplied files, and the correct approach is to identify the anomalous artifact.

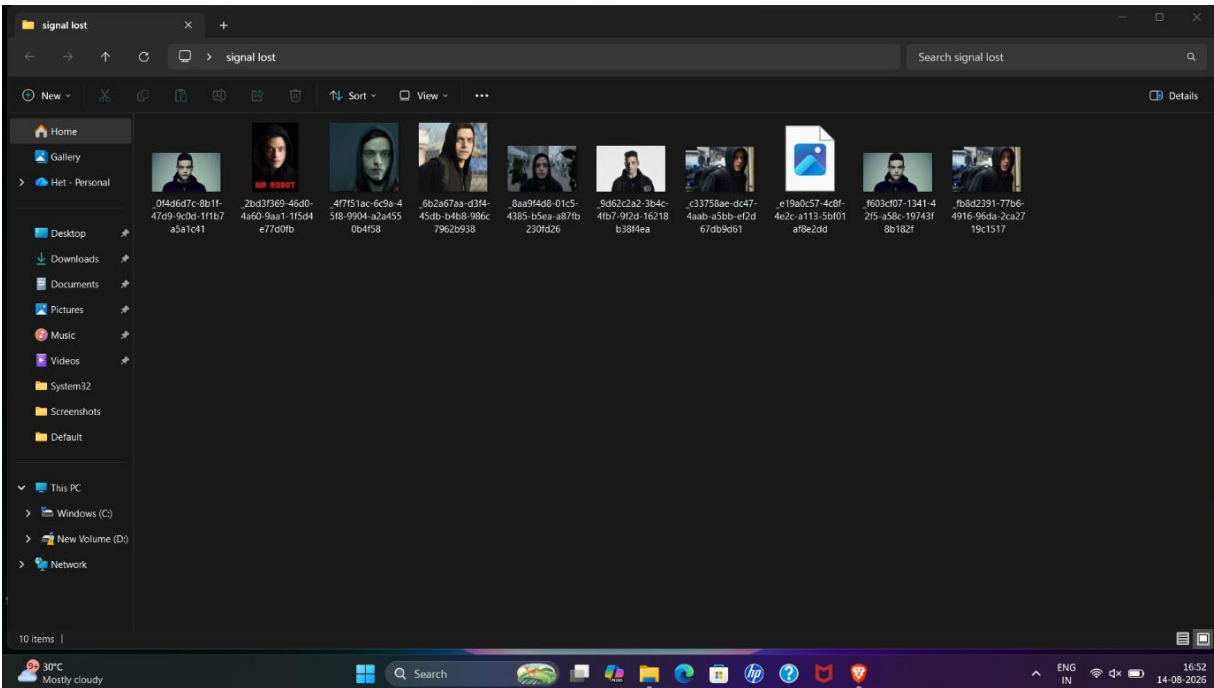


Figure 2 — Initial directory containing the recovered challenge artifacts. The bundle contains multiple image files and one PDF-like document.

Evidence inventory

Evidence class	Observed artifact	Role in investigation
Image set	Multiple UUID-style filenames	Candidate surveillance stills
PDF	download_unlocked.pdf	Contains the archive-key clue
Suspicious image	_2bd3f369-46d0-4a60-9aa1-1f5d4e77d0fb.jpg	Contains embedded signal blob
Recovered archive	dispatch.7z	Encrypted container holding final evidence

3. Initial Access to the Recovered PDF

The recovered PDF was initially password protected. The screenshots show exploratory use of CyberChef's Magic operation to identify the file type and an attempt to open the PDF, which confirmed that a password was required.

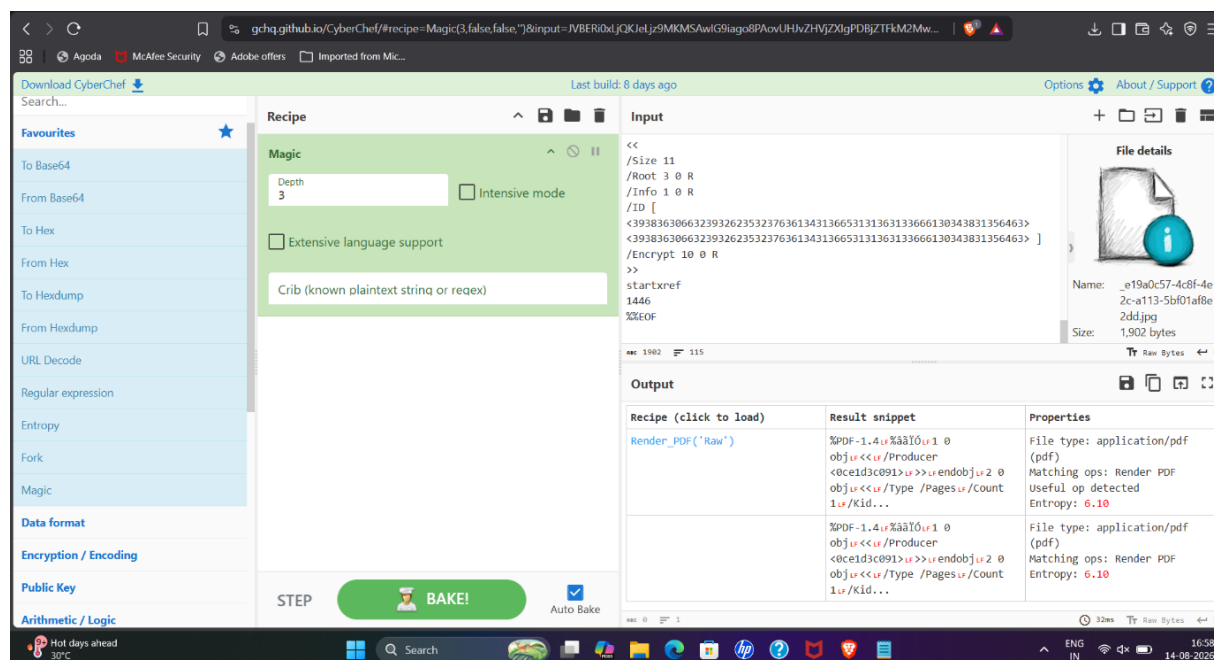


Figure 3 — CyberChef Magic identified the input as a PDF (application/pdf).

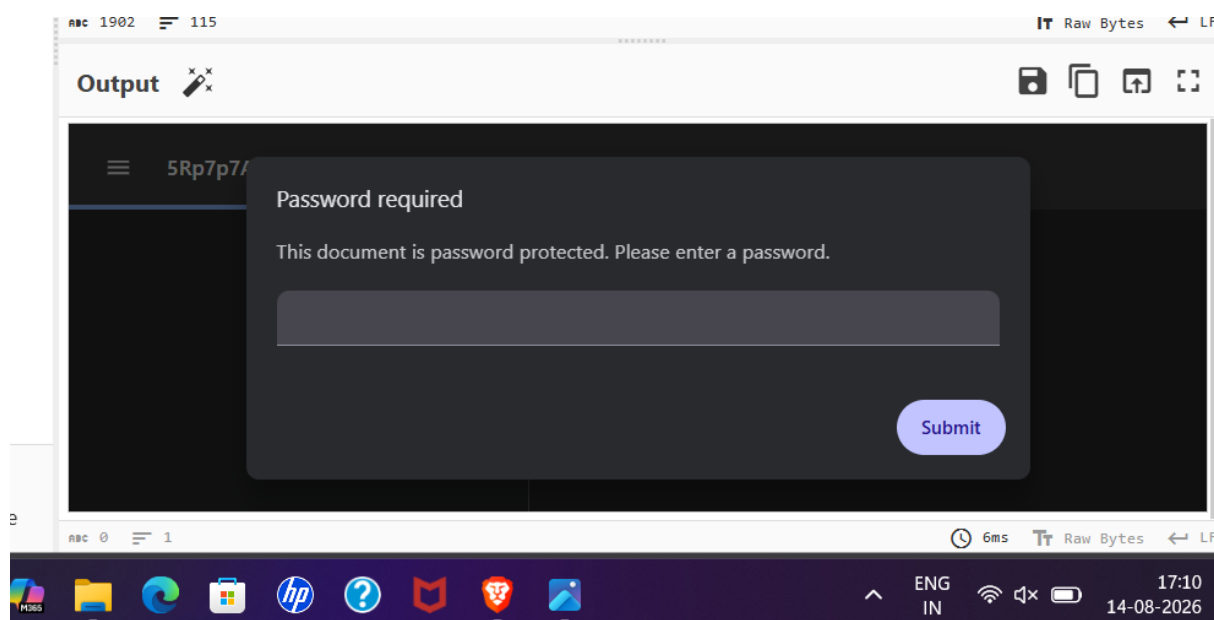


Figure 4 — The PDF viewer requested a password, confirming that the document was encrypted/protected.

FORENSIC NOTE

The challenge evidence was unlocked using an online PDF utility (I love pdf site) during the solve. That is acceptable as a CTF shortcut, but it is not a preferred forensic-preservation technique for real investigations because evidence should remain under controlled custody.

4. Recovering the Archive Key

After the PDF was made readable, it contained a short instruction stating that the archive key was not written plainly and that the signal should be decoded three times. The visible payload was then processed with three Base64 decoding operations.

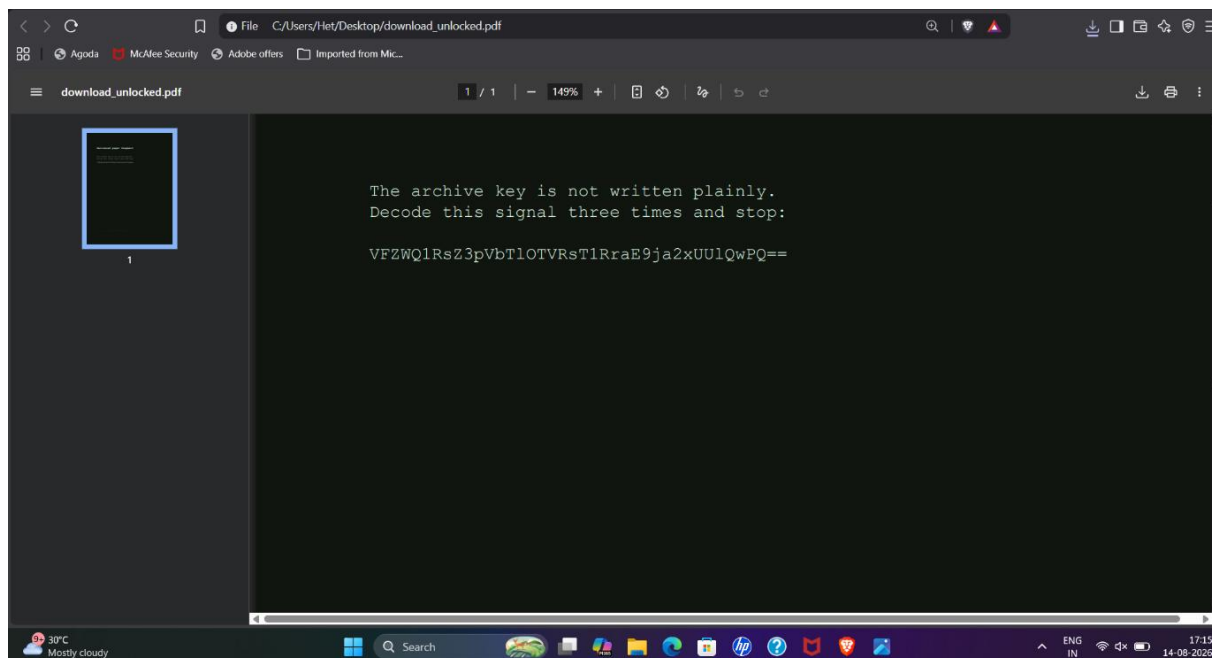


Figure 5 — Recovered PDF clue. It explicitly instructs the analyst to decode the supplied signal three times.

Used FROM BASE 64 operation thrice as said.

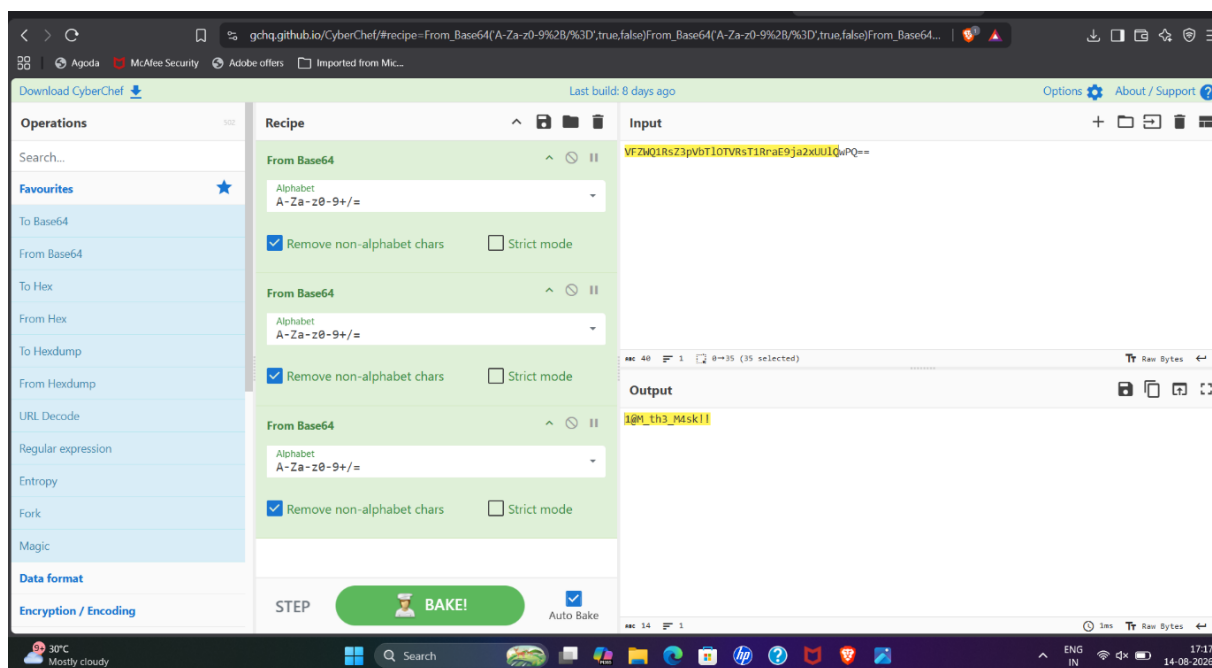


Figure 6 — CyberChef recipe applying From Base64 three times. The final output reveals the archive key.

Recovered archive key: 1@m_th3_M4sk!!

The screenshot shows the key as `1@m\_th3\_M4sk!!`. This value was then used to access the encrypted 7z archive.

5. Identifying the Suspicious Image

The supplied image set contained visually different photographs, but the decisive indicator was not the appearance of the photographs. The investigation focused on file content. An anomalous image was opened as raw text and displayed a large amount of non-image data, indicating that the file contained additional embedded material.

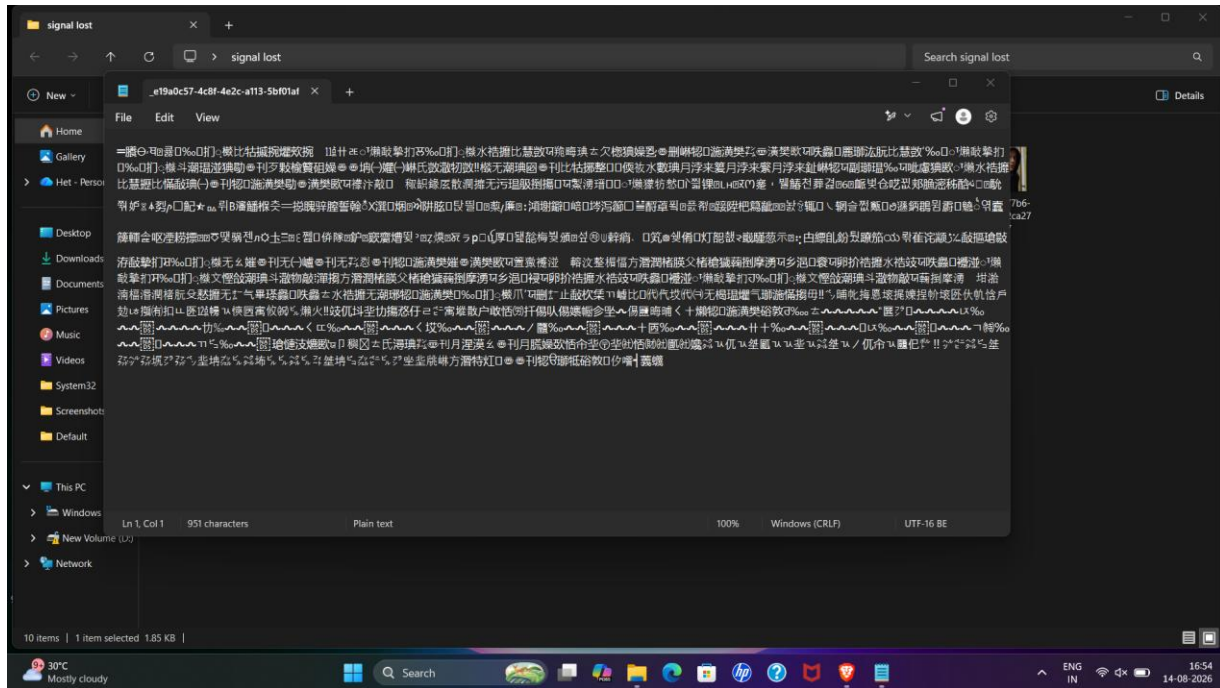


Figure 7 — Raw-text view of an image file showing a large non-image data region. This was an early indicator that the artifact warranted deeper inspection.

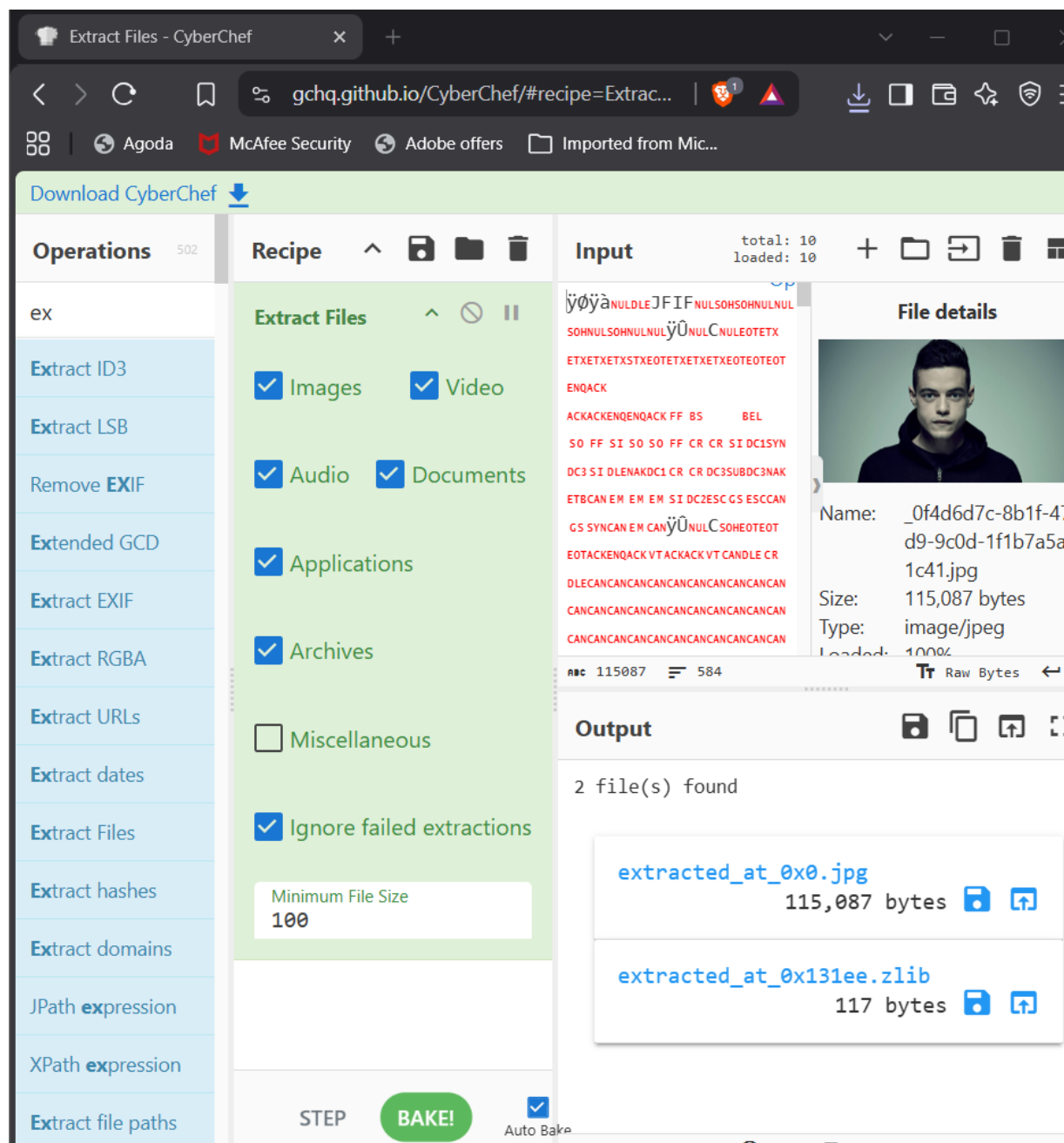


Figure 8 — CyberChef Extract Files operation against the suspicious JPEG. It identified an additional embedded zlib object.

The exploratory CyberChef path detected an embedded zlib object. This was useful for confirming that the JPEG was carrying nonstandard content, but it was not the cleanest path to the final dispatch.

6. Exploratory Analysis / Discarded Leads

Not every attempted transformation contributed directly to the final solve. These screenshots are retained because they document the investigation process and show how false leads were tested and rejected.

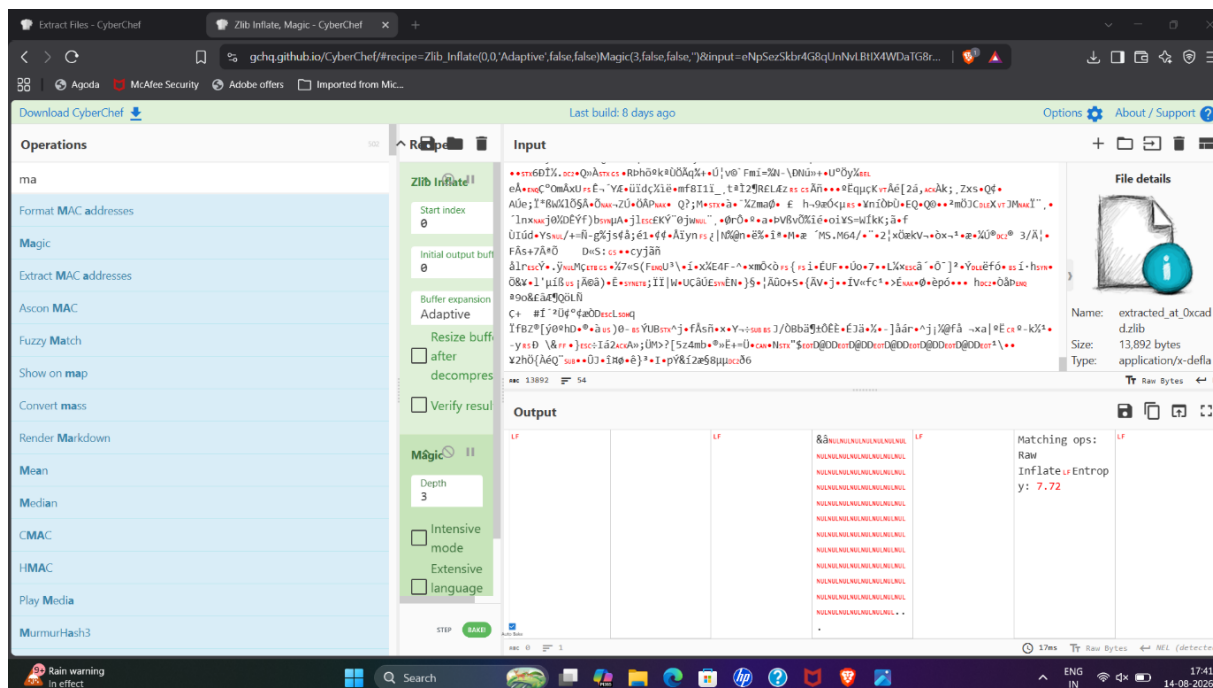


Figure 9— Attempt to inflate an extracted zlib object. The output did not immediately yield the dispatch.

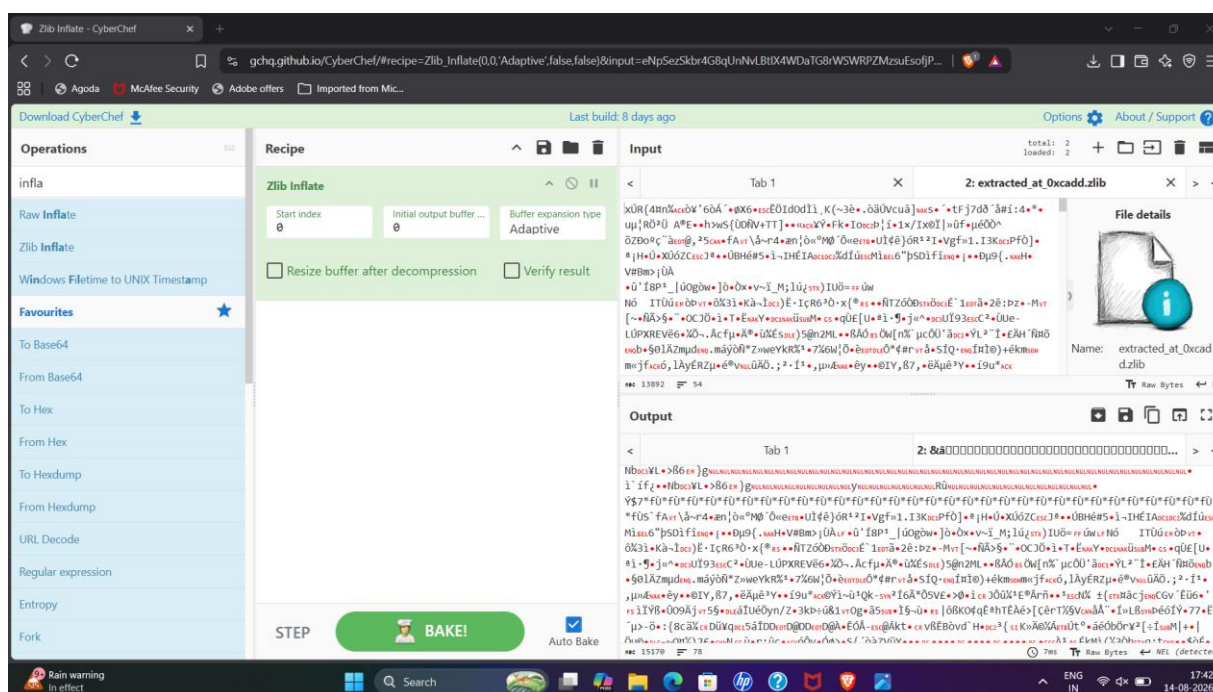


Figure 10— Further zlib-inflate experimentation. The result remained non-readable/binary-like.

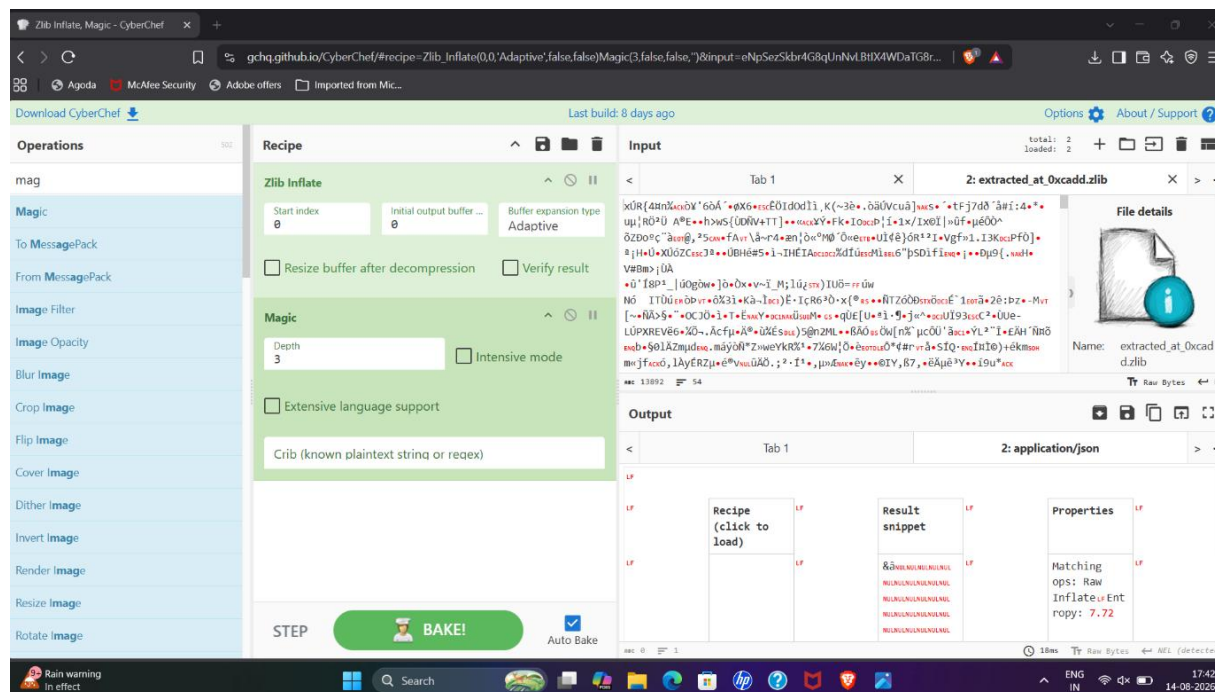


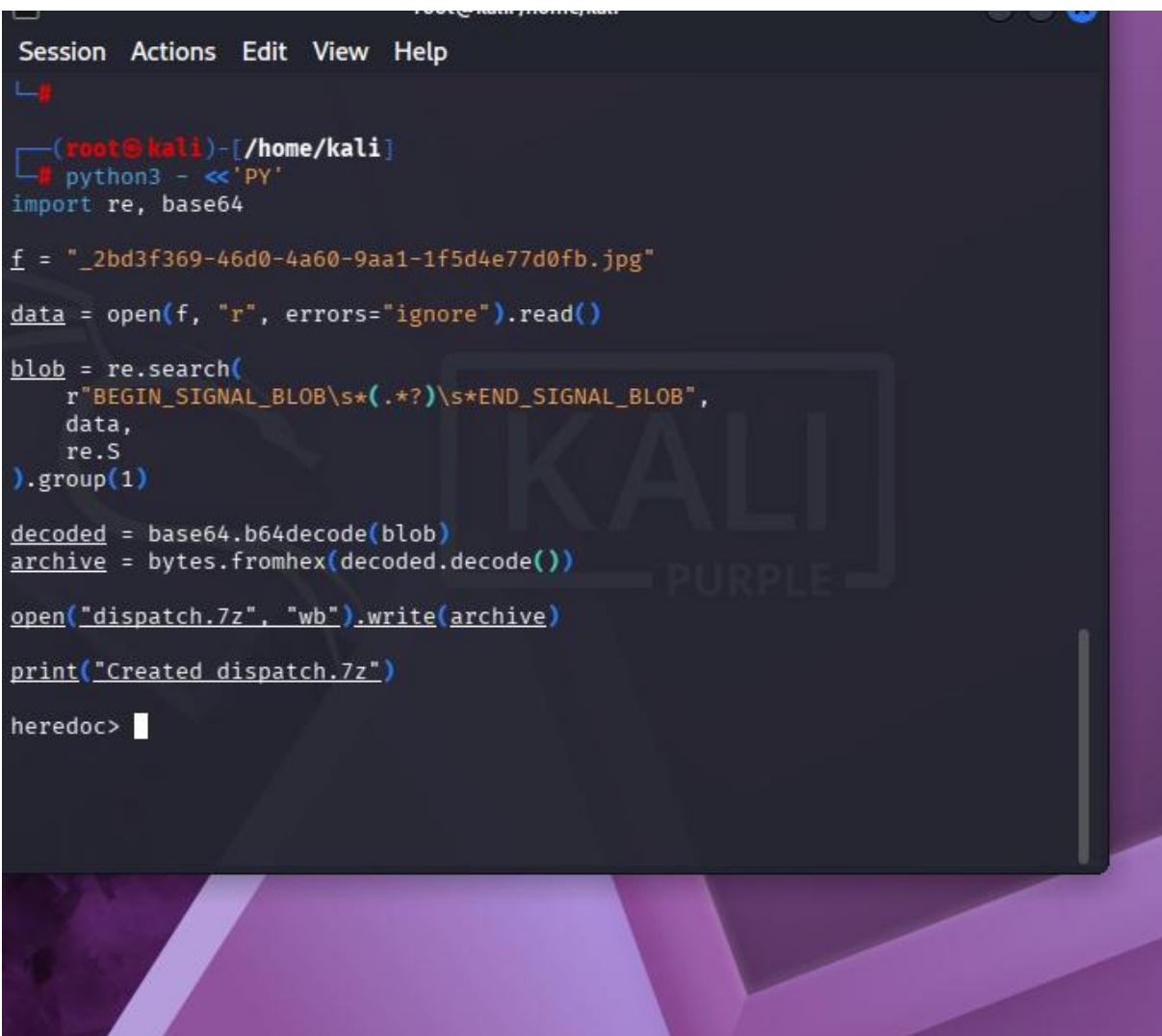
Figure 11 — Magic detection after decompression. The detected content did not provide the final dispatch.

DECISION POINT

The investigation was simplified by returning to the strongest structural clue: the explicit `BEGIN_SIGNAL_BLOB / END_SIGNAL_BLOB` markers and decoding that payload directly.

7. Direct Extraction of the Embedded Signal

The successful route used a short Python script to read the suspicious JPEG, locate the embedded signal markers, extract the content between them, Base64-decode it, interpret the result as hexadecimal, and write the resulting bytes to a 7z archive.



```

Session  Actions  Edit  View  Help

#

(root@kali)-[/home/kali]
# python3 - <<'py'
import re, base64

f = "_2bd3f369-46d0-4a60-9aa1-1f5d4e77d0fb.jpg"

data = open(f, "r", errors="ignore").read()

blob = re.search(
    r"BEGIN_SIGNAL_BLOB\s*(.*?)\s*END_SIGNAL_BLOB",
    data,
    re.S
).group(1)

decoded = base64.b64decode(blob)
archive = bytes.fromhex(decoded.decode())

open("dispatch.7z", "wb").write(archive)

print("Created dispatch.7z")

heredoc>

```

Figure 12 — Python extraction script locating `BEGIN_SIGNAL_BLOB` and `END_SIGNAL_BLOB`, then Base64-decoding and converting the result from hexadecimal into archive bytes.

```

import re, base64

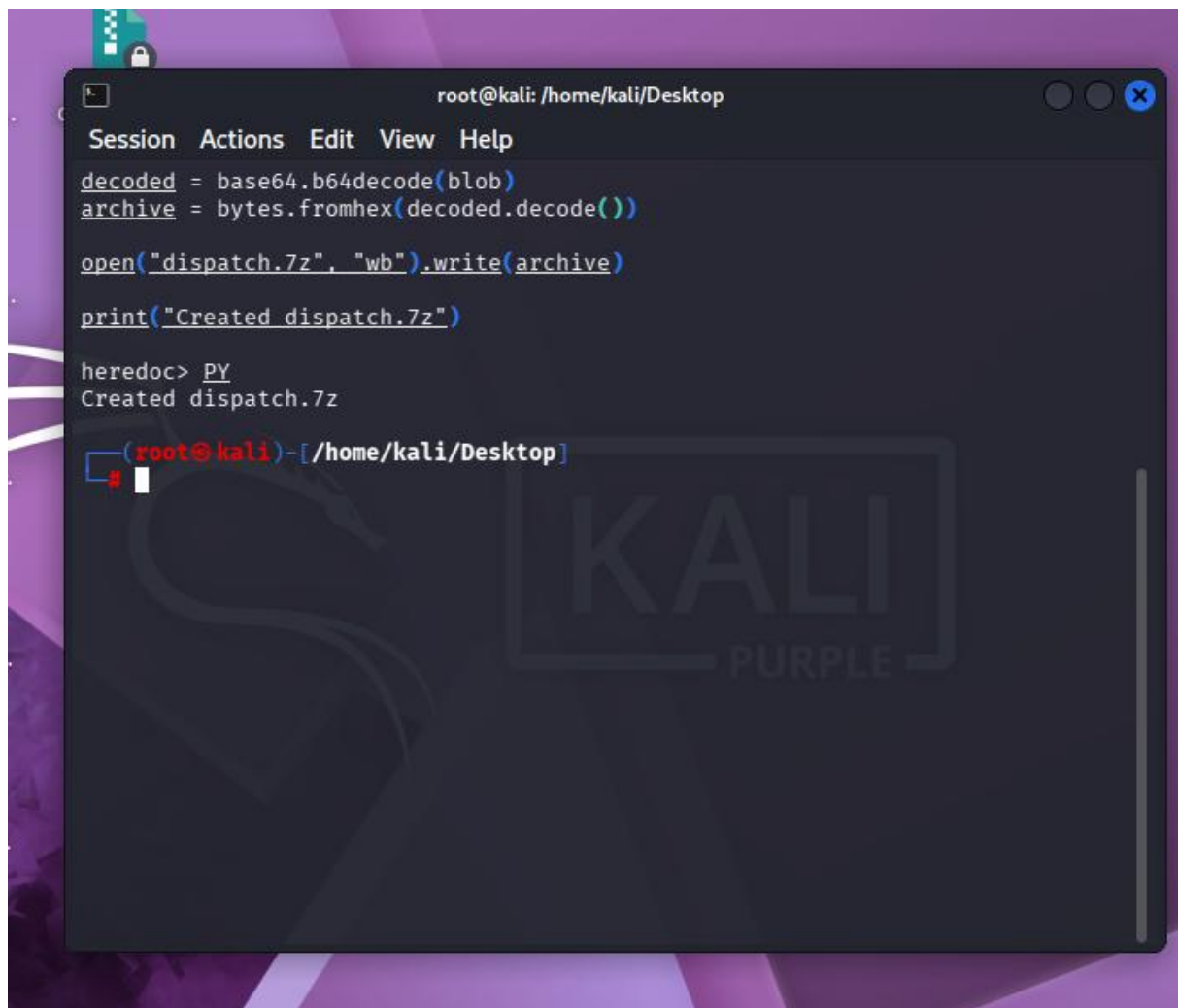
f = "_2bd3f369-46d0-4a60-9aa1-1f5d4e77d0fb.jpg"
data = open(f, "r", errors="ignore").read()

blob = re.search(
    r"BEGIN_SIGNAL_BLOB\s*(.*?)\s*END_SIGNAL_BLOB",
    data, re.S
).group(1)

decoded = base64.b64decode(blob)
archive = bytes.fromhex(decoded.decode())

open("dispatch.7z", "wb").write(archive)

```



```
root@kali: /home/kali/Desktop
Session Actions Edit View Help
decoded = base64.b64decode(blob)
archive = bytes.fromhex(decoded.decode())

open("dispatch.7z", "wb").write(archive)

print("Created dispatch.7z")

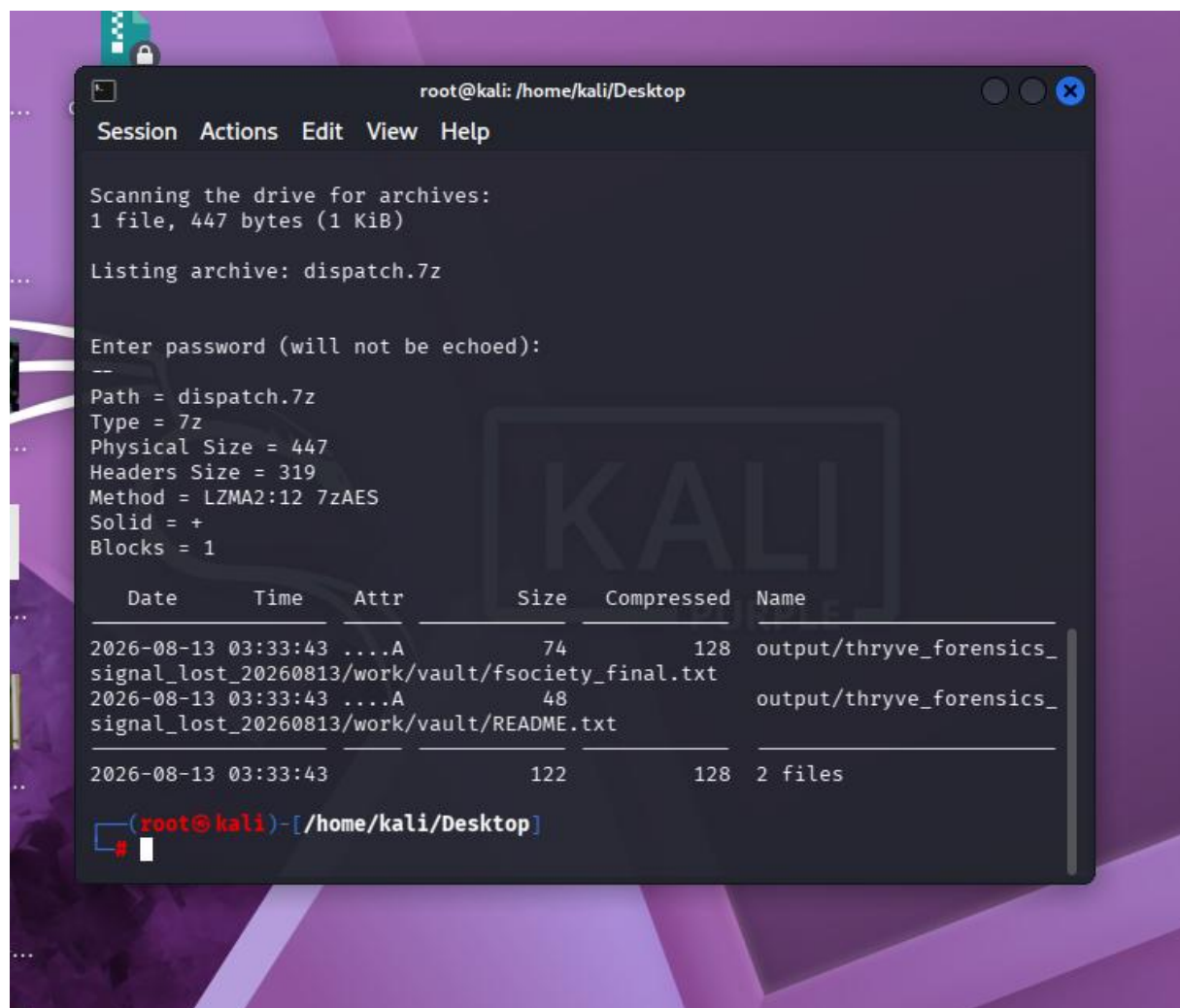
heredoc> PY
Created dispatch.7z

(root@kali)-[/home/kali/Desktop]
#
```

Figure 13 — Successful execution of the extraction script. The terminal confirms that `dispatch.7z` was created.

8. Archive Analysis

The resulting file was identified as a 7z archive. Using the recovered key, the archive could be listed successfully. It contained two files: `README.txt` and `fsociety_final.txt`.



```

root@kali: /home/kali/Desktop
Session Actions Edit View Help

Scanning the drive for archives:
1 file, 447 bytes (1 KiB)

Listing archive: dispatch.7z

Enter password (will not be echoed):
--
Path = dispatch.7z
Type = 7z
Physical Size = 447
Headers Size = 319
Method = LZMA2:12 7zAES
Solid = +
Blocks = 1

  Date       Time       Attr      Size   Compressed  Name
-----
2026-08-13 03:33:43 ....A        74        128 output/thryve_forensics_
signal_lost_20260813/work/vault/fsociety_final.txt
2026-08-13 03:33:43 ....A         48          output/thryve_forensics_
signal_lost_20260813/work/vault/README.txt
-----
2026-08-13 03:33:43          122        128 2 files

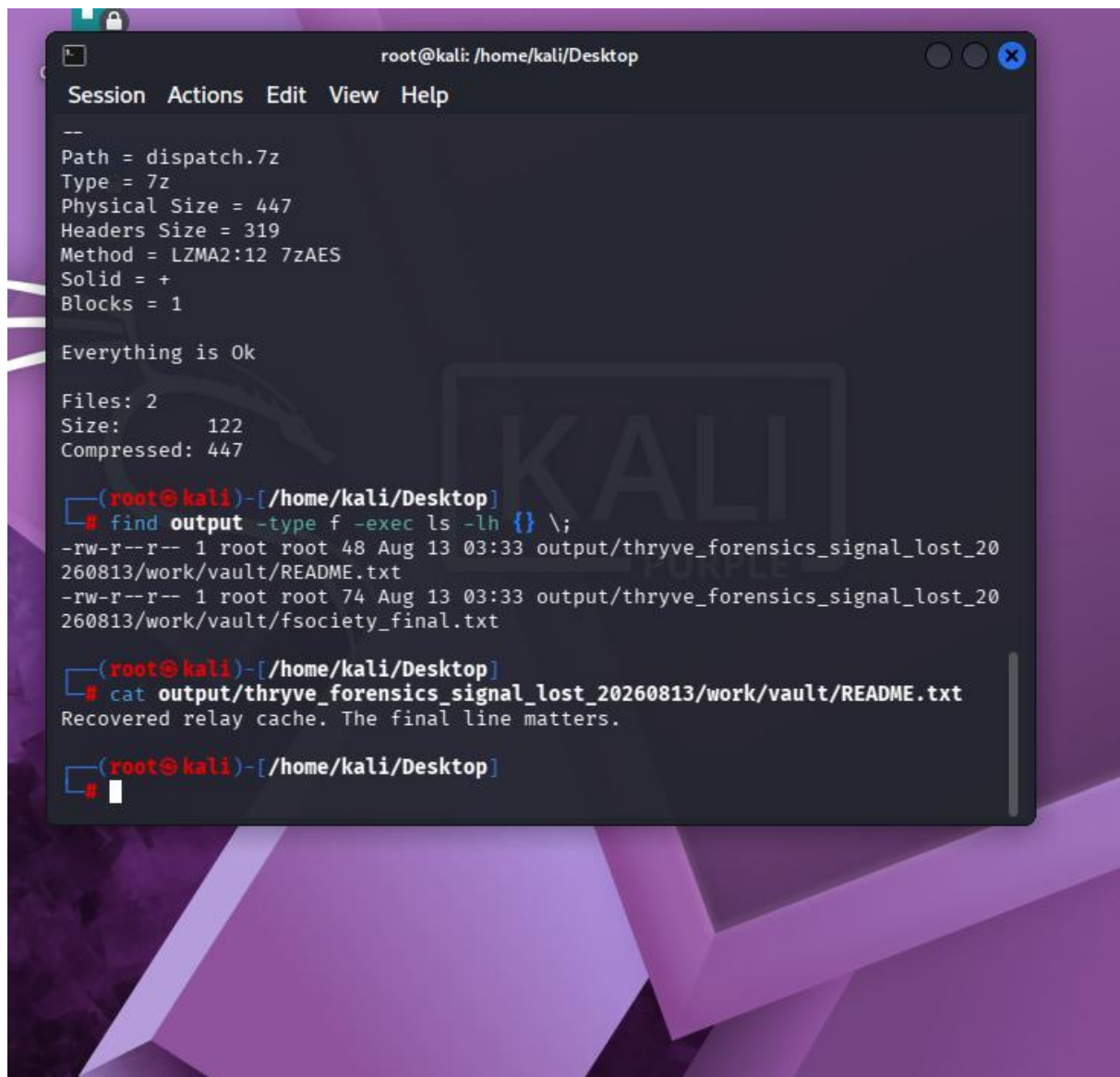
(root@kali)-[/home/kali/Desktop]

```

Figure 14 — 7z archive listing. The archive is recognized as a 7z container and contains `README.txt` and `fsociety_final.txt`.

The archive metadata shown in the terminal included a 7z container using LZMA2 compression and AES encryption. The archive contents were small and directly relevant to the final stage.

9. Evidence Extraction & Final Instruction



```
root@kali: /home/kali/Desktop
Session Actions Edit View Help
--
Path = dispatch.7z
Type = 7z
Physical Size = 447
Headers Size = 319
Method = LZMA2:12 7zAES
Solid = +
Blocks = 1

Everything is Ok

Files: 2
Size:      122
Compressed: 447

(root@kali)-[/home/kali/Desktop]
# find output -type f -exec ls -lh {} \;
-rw-r--r-- 1 root root 48 Aug 13 03:33 output/thryve_forensics_signal_lost_20
260813/work/vault/README.txt
-rw-r--r-- 1 root root 74 Aug 13 03:33 output/thryve_forensics_signal_lost_20
260813/work/vault/fsociety_final.txt

(root@kali)-[/home/kali/Desktop]
# cat output/thryve_forensics_signal_lost_20260813/work/vault/README.txt
Recovered relay cache. The final line matters.

(root@kali)-[/home/kali/Desktop]
#
```

Figure 15 — Archive extraction and README review. The README states: 'Recovered relay cache. The final line matters.'

The README established the final extraction rule: the analyst should inspect the final line of the recovered dispatch rather than attempting additional transformations.

```
cat output/thryve_forensics_signal_lost_20260813/work/vault/README.txt
Recovered relay cache. The final line matters.
```


10. Final Dispatch & Flag

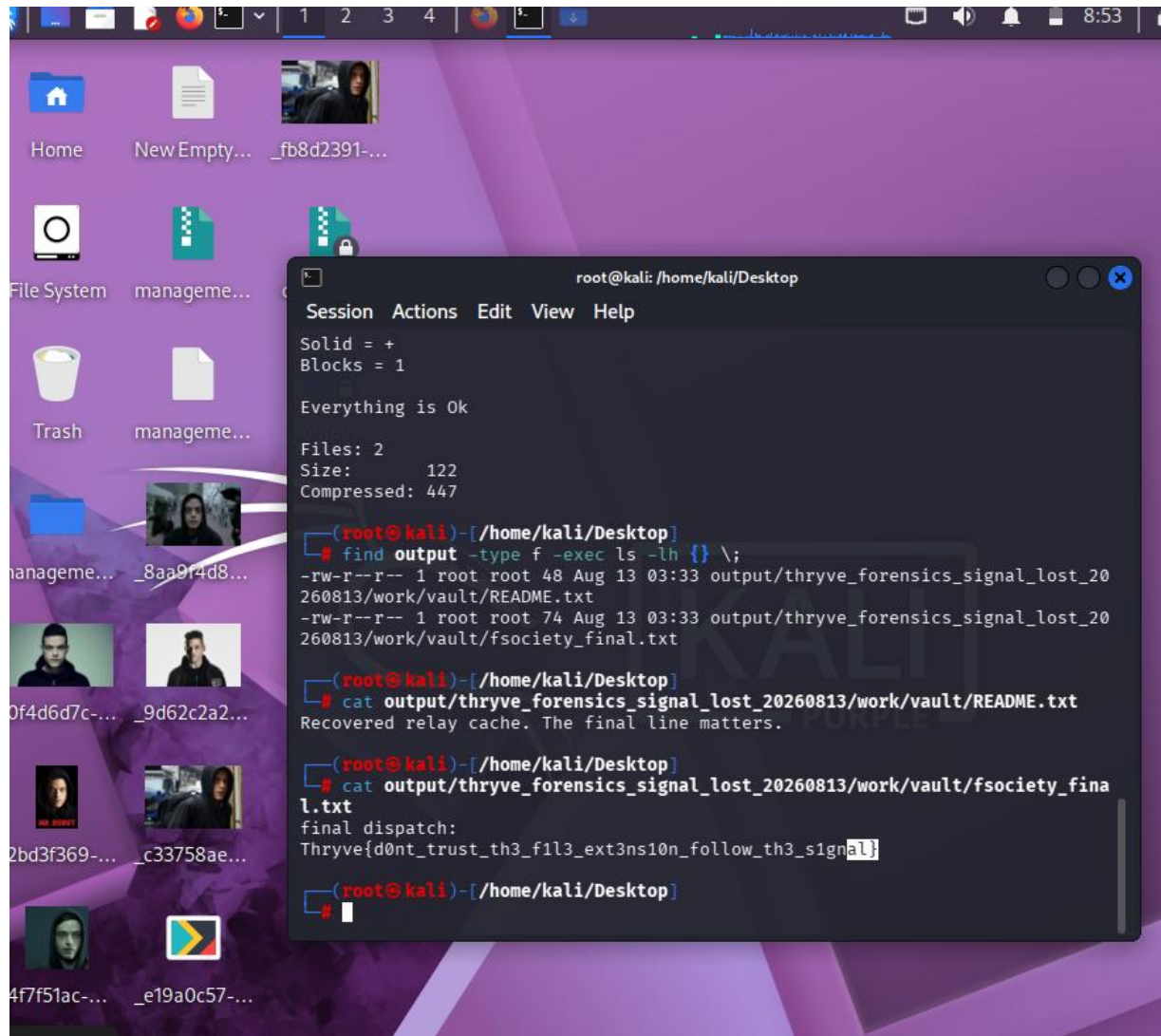


Figure 16 — Final terminal evidence. The extracted `fociety_final.txt` contains the final dispatch and the accepted flag.

RECOVERED FLAG

Thryve{d0nt_trust_th3_f1l3_ext3ns10n_follow_th3_s1gnal}

The flag directly matches the challenge's required `Thryve{...}` format and was recovered from the final dispatch file. No further decoding was necessary.

11. Investigation Timeline

Step	Stage	Finding / action
01	Challenge triage	Read the DFIR prompt and identified the need to locate one anomalous surveillance still.
02	Artifact review	Reviewed the supplied files and isolated the suspicious image.
03	Content inspection	Found embedded/non-image data and

04	Key recovery	the signal blob structure.
05	Payload extraction	Opened the recovered PDF clue and decoded the signal three times to obtain the archive key.
06	Archive access	Extracted BEGIN_SIGNAL_BLOB / END_SIGNAL_BLOB and converted the payload into dispatch.7z.
07	Evidence extraction	Used the recovered key to list the encrypted 7z archive.
08	Final recovery	Recovered README.txt and fsociety_final.txt.
		Followed the README instruction and extracted the flag from the final dispatch line.

12. DFIR Lessons Learned

- Do not trust file extensions. A file named as an image can contain additional data or another payload.
- When a challenge says one artifact is different, compare structure and content rather than relying only on visual appearance.
- Strings/raw-text inspection can reveal embedded markers that ordinary image viewers will never show.
- Track the transformation chain. Every recovered artifact should have a clear parent and decoding step.
- Exploratory tools such as CyberChef are useful for hypothesis testing, but a short deterministic script can provide a cleaner and reproducible extraction path.
- In real forensic work, avoid online evidence-processing services unless explicitly authorized; preserve originals and work from verified copies.

13. Evidence Chain / Reproducibility

```

Recovered PDF clue
    ↓
Three × Base64 decoding
    ↓
Archive key: 1@M_th3_M4sk!!
    ↓
Suspicious JPEG
    ↓
Extract BEGIN_SIGNAL_BLOB ... END_SIGNAL_BLOB
    ↓
Base64 decode
    ↓
Hex → binary
    ↓
dispatch.7z
    ↓
README.txt + fsociety_final.txt
    ↓
Final line
    ↓
Thryve{d0nt_trust_th3_f1l3_ext3ns10n_follow_th3_s1gnal}

```


14. Analyst Notes

This report intentionally distinguishes the successful evidence path from exploratory attempts. Screenshots that did not directly produce the flag are retained under the exploratory-analysis section instead of being presented as successful steps. This preserves an honest investigation record while keeping the primary solve reproducible.

END OF REPORT